Lab 12:

Reading and writing files:

CreateFile Function:

The CreateFile function either creates a new file or opens an existing file. If successful, it returns a handle to the open file; otherwise, it returns a special Constant name INVALID_HANDLE_VALUE.

Here is the prototype:

```
Call args:  EDX contains the offset of a filename

Return arg: EAX contains a valid file handle if the operation was successful.
            Otherwise, EAX contains INVALID_HANDLE_VALUE.
```

Example:

```
      mov  edx,OFFSET filename
      call CreateOutputFile
      mov  filehandle, EAX
```

ReadFile Function

The ReadFile function reads text from an input file. Here is the prototype:

```
Call args:  EAX = an open file handle
            EDX = offset of the input buffer
            ECX = maximum number of bytes to read

Return arg:
If CF = 0, EAX contains the number of bytes read.
If CF = 1, EAX contains a system error code. Call WriteWindowsMsg
to get a text representation of the error.
```

Example:

```
.data
BUFSIZE = 5000
buffer BYTE BUFSIZE DUP(?)
bytesRead DWORD ?

.code
      mov  eax,fileHandle
      mov  edx,OFFSET buffer
      mov  ecx,BUFSIZE
      call ReadFromFile
      jc   show_error_message
      mov  bytesRead,eax
```

WriteFile Function

The WriteFile function writes data to a file, using an output handle. The handle can be the

screen buffer handle, or it can be one assigned to a text file. The function starts writing data to

the file at the position indicated by the file's internal position pointer. After the write operation has

been completed, the file's position pointer is adjusted by the number of bytes actually written. Here is the function prototype:

```
Call args:  EAX = an open file handle
            EDX = offset of the buffer
            ECX = maximum number of bytes to write

Return arg:
If CF = 0, EAX contains the number of bytes written.
If CF = 1, EAX contains a system error code. Call WriteWindowsMsg
to get a text representation of the error.

Example:
.data
BUFSIZE = 5000
buffer BYTE BUFSIZE DUP(?)
bytesWritten DWORD ?

.code
      mov  eax,fileHandle
      mov  edx,OFFSET buffer
      mov  ecx,BUFSIZE
      call WriteToFile
      jc   show_error_message
      mov  bytesWritten,eax
```

CloseFile Function

```
Call args:  EAX contains an open file handle

Return arg: EAX is nonzero if the operation was successful

Example:

      mov  eax,fileHandle
      call CloseFile
```

MWrite

Writes a string literal (no terminal null) to standard output.

Parameter:

text:REQ - A string literal.

Example:

```
mWrite "Hello World"
```

Dependencies:

Causes a call to the WriteString procedure.

Tasks:

1. Write an assembly program that takes input of filename to be created and creates file of that name. After that write the following text in file using your program:

   Hello my name is {your name}. My hometown is {your hometown}. My roll number is {your rollnumber}. I love assembly language and I don't mind repeating it.
2. Write an assembly program opens the above created filed and does the following:
   a. Read the file
   b. Calculate number of vowels and consonants in the above file content.
   c. Count number of words in the above file
   d. Extract your name, roll number and hometown from filecontent (you may use hardcoded indexes to do that).
   e. Display them on console.
3. Make a simple ball hit game where ball hits with the wall increase the score and save this score in descending order in the file and also read the score and display on console.
4. Write assembly language program that:

   a. Opens a text file and print its content on console.
   b. Writes a string message in text file.
   c. Appends the text file.
   d. Closes the text file.

Submission Instructions:

- Write your name, roll no and section on top of your code file
- Do all of your code work in procedures and just call that procedure in main to execute the code.

  Eg:

```
Q1  proc
;code
ret
Q1 endP
```

- Submit only one .asm file (Format: i22-1234_LAB13.asm